

Key decisions

My main positioning and stand was to recommend a 90-day pilot to the hypothetical customer to custom build one of their low-risk workflows with Devin and everything about the tradeoffs and decisions were shaped around this recommendation.

I chose a greenfield, non-transactional Feature-Flag Change-Control Plane. The two-hour proof tests Devin on reviewable code and delivery artifacts without putting PII, money, or real production behavior at risk. I kept KYC and refunds on Power Apps initially because their compliance and operational blast radius could block an FDE POC or pilot, and Power Apps already provides platform controls this prototype leaves out. From my experience, this is major risk when working with IT and Cybersec on getting approvals, access etc. I screened candidate workflows against value, timebox, blast radius, and reusable controls:

Candidate	Decision
KYC queue or refunds	High value; high blast radius (PII/money); exclude
Risk rules or reconciliation	High value; deep domain/integrations; later candidate
Read-only transaction explorer	Low value; easy, weak workflow proof; exclude
Feature-flag change control	High value; moderate, virtual/mock; chosen

A developer proposes a virtual flag change. A separate release approver approves or rejects it, then applies or rolls it back; an auditor is read-only. This enforces segregation of duties. Real flags can affect payments, access, or risk, so these flags stay virtual and non-transactional. State machine: `DRAFT` -> `PENDING_APPROVAL` -> `APPROVED` -> `APPLIED` -> `ROLLED_BACK`, or `PENDING_APPROVAL` -> `REJECTED`. Each valid transition writes an audit event in the same database transaction.

Architecture: the browser presents the workflow, while the server owns identity, permissions, and state changes. PostgreSQL stores requests, approvals, and audit history. The local adapter stores simulated flag state in the same database, so provider state, workflow state, and each audit event commit together. This makes the proof deterministic, but gives up realism about an external provider. I chose Next.js and TypeScript to keep the UI and server logic in one codebase, avoiding separate frontend and API deployments within the timebox. The prototype proves the workflow and application controls and is not production parity with Power Apps hypothetical internal dashboard for flags. Real enterprise identity, external-provider integration, Power Apps governance equivalents, durable audit retention, multi-tenancy, and deployment remain out of scope. Docker and GitHub CI keep the app, database, and tests repeatable.

I ran a separate Claude `/deep-research` workflow on Power Apps and fintech fit; its notes and sources are committed under `/docs`. I also copied selected local workflow guidance into the repo, including `AGENTS.local.md`, multi-agent code review, a book-based business-storytelling skill, and an AI Slop Remover.